

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3

«Разработка одностраничного веб-приложения (SPA) с использованием
фреймворка Vue.JS»

Выполнил:
Демьянов Фёдор

Группа: J3214

Проверил:
Добряков Д. И.

Санкт-Петербург

2023 г.

1. Цель работы

Перевести веб-сервис бронирования космических путешествий «NovaTransit», спроектированный в рамках лабораторных работ № 1 и № 2, на компонентную архитектуру фреймворка Vue 3. Разработать одностраничное веб-приложение (SPA) с сохранением визуального стиля и доступности, подключить клиентскую маршрутизацию через Vue Router с защитой закрытых разделов, централизовать сетевые запросы к локальному REST API через библиотеку Axios, разделить монолитную верстку на независимые переиспользуемые компоненты и вынести повторяющуюся логику в отдельные composables.

2. Архитектура приложения и декомпозиция компонентов

2.1. Стек технологий и организация проекта

Для перехода от набора разрозненных HTML-страниц к единому приложению был выбран актуальный стек инструментов фронтенд-разработки:

- **Vue 3 (Composition API):** логика компонентов построена на синтаксисе `<script setup>`. Это делает код компактнее, избавляет от громоздких объектов `data` и `methods`, а также упрощает работу с реактивными ссылками `ref` и `computed`.
- **Vite:** быстрый сборщик, обеспечивающий мгновенный запуск сервера разработки за счет нативной отдачи ES-модулей в браузер.
- **Vue Router 4:** обеспечивает бесшовные переходы между разделами сервиса без физической перезагрузки страницы браузером и контролирует права доступа.
- **Axios:** специализированный HTTP-клиент, пришедший на смену самодельному модулю на базе Fetch API.
- **Bootstrap 5 и Bootstrap Icons:** использованы для базовой сетки, системных отступов и адаптивности. Кастомный визуал космического терминала, неоновые эффекты и CSS-переменные тем сохранены в файле `style.css`.
- **SVG-спрайт (AppSvgSprite.vue):** вся векторная графика (логотип, ракета, стрелки, маркеры списков) вынесена в единый скрытый SVG-контейнер, что исключает ошибки подгрузки внешних файлов браузером при смене URL.
- **json-server:** локальный сервер, имитирующий серверную базу данных на основе файла `db.json`.
- **NASA APOD API:** сторонний публичный интерфейс для получения астрономического снимка дня.

2.2. Поставленные задачи и их практическая реализация

В рамках выполнения лабораторной был сформирован и последовательно реализован комплекс задач:

- **Развертывание среды сборки и SPA-инфраструктуры:**
 - Инициализирован проект на базе сборщика Vite с шаблоном Vue 3 и синтаксисом `<script setup>` (Composition API).
 - Подключены библиотеки Vue Router 4, Axios, Bootstrap 5 и Bootstrap Icons.
 - Интегрирован глобальный компонент векторного спрайта `AppSvgSprite.vue`, устранивший сетевые сбои подгрузки SVG-графики на вложенных маршрутах браузера.
- **Декомпозиция пользовательского интерфейса на компоненты:**
 - Пять монолитных HTML-страниц разделены на полноэкранные представления верхнего уровня (`src/views/`) и дочерние переиспользуемые блоки (`src/components/`).
 - Настроена однонаправленная передача данных через входные параметры `props` и подписка на пользовательские действия через события `emit`.
 - Формы ввода унифицированы с помощью директивы двустороннего связывания `v-model`.
- **Организация клиентской маршрутизации и контроля доступа:**
 - Настроена карта маршрутов во Vue Router 4 с применением динамического импорта (`import()`) для отложенной загрузки страниц по требованию (Lazy Loading).
 - Реализован навигационный барьер `router.beforeEach`, блокирующий несанкционированный переход к личному кабинету `/dashboard` и автоматически перенаправляющий авторизованных кадетов из гостевых терминалов `/login` и `/register`.
- **Централизация сетевого взаимодействия через Axios:**
 - Создан базовый экземпляр Axios с единой конфигурацией `baseURL`: `http://localhost:3000` и системными заголовками.
 - Логика обращений к серверной базе данных `db.json` разделена на предметные модули: реестр полетов (`toursApi`), профили исследователей (`usersApi`) и бронирование билетов (`bookingsApi`).

- **Инкапсуляция бизнес-логики в Composables:**

- **useTheme:** реализовано реактивное управление темами оформления *Eclipse* и *Nova* с поддержкой системных настроек ОС **prefers-color-scheme** и кэшированием в **localStorage**.
- **useAuth:** реализовано централизованное хранилище состояния сессии, регистрация новых исследователей с генерацией персонального шифра, вход в систему и частичное сохранение систем каюты методом **PATCH**.
- **useNasaApod:** построен отказоустойчивый сетевой шлюз к внешнему REST API NASA с каскадным переключением (сессионный кэш $\rightarrow$ прямой запрос $\rightarrow$ CORS-прокси $\rightarrow$ автономный оффлайн-архив).

2.3. Иерархия компонентов и декомпозиция страниц

В соответствии с правилами компонентного подхода все пять страниц сервиса были разделены на представления верхнего уровня (**views**) и дочерние компоненты (**components**), решающие конкретные интерфейсные задачи:

1. Главная страница (**HomeView.vue**):

- **TheNavbar.vue** - сквозная навигационная панель.
- **PlanetPreview.vue** - принимает через входной параметр **destination** выбранное направление и реактивно переключает фоновые слои планет, обновляя текстовые параметры силы тяжести и времени полета.
- **TourSearchPanel.vue** - форма фильтрации по орбите, длительности и бюджету. При выборе тарифов VIP и Premium активирует декоративное свечение рамки.
- **TourCard.vue** - отдельная карточка миссии, отображающая технические спецификации и формирующая ссылку перехода на страницу тура.

2. Детальная страница экспедиции (**TourDetailsView.vue**):

- **TourGallery.vue** - слайдер фотографий с горизонтальным смещением через свойство **transform: translateX()**, поддержкой автоматической прокрутки каждые 5 секунд и паузой при наведении курсора мыши.
- **TourSpecs.vue** - сетка из четырех ключевых физических характеристик (дистанция, гравитация, перегрузки, радиационный фон).
- **TourTimeline.vue** - визуальный график миссии с отметками времени и разделительными направляющими.
- **BookingModal.vue** - диалоговое окно бронирования, анимированное компонентом **<Transition>**. Производит динамический расчет итоговой цены при

включении дополнительных услуг (шлюзование EVA и внешняя съемка челнока 8K) и отправляет оформленный билет на сервер.

3. Личный кабинет исследователя (**DashboardView.vue**):

- **ActiveFlightCard.vue** - карточка ближайшей подтвержденной миссии.
- **UserProfileCard.vue** - персональное досье кадета с кодом допуска, классом и кнопкой завершения сессии.
- **TrainingProgress.vue** - индикаторы прогресса предполетной подготовки.
- **CabinControls.vue** - тумблеры систем каюты (гравитация, питание, автозатемнение), отправляющие частичные обновления на сервер.
- **NasaApodCard.vue** - телеметрический шлюз к космической базе NASA с адаптивным выводом видеороликов или широкоформатных снимков.
- **FlightHistoryTable.vue** - таблица всех совершенных и запланированных полетов исследователя.
- **TicketModal.vue** - всплывающее окно цифрового сертификата со штрихкодом и серийным номером.

4. Терминалы входа и регистрации (**LoginView.vue**, **RegisterView.vue**):

- Содержат изолированные формы с клиентской валидацией обязательных полей, проверкой длины ключа доступа, подтверждением регламента полетов и выводом серверных ошибок.

2.4. Поток данных и взаимодействие компонентов

В приложении выстроена однонаправленная передача данных, дополненная реактивными composables для глобального состояния:

- **Передача данных сверху вниз (props)**: родительские страницы передают во вложенные компоненты необходимые срезы данных (например, объект характеристик миссии в **TourSpecs** или полетные манифесты в **FlightHistoryTable**).
- **Передача событий снизу вверх (emit)**: дочерние компоненты оповещают родителей о действиях пользователя через механизм событий (**close** для модальных окон, **update:filter** и **search** для панели поиска, **notify** для вызова системных уведомлений).
- **Двустороннее связывание (v-model)**: используется внутри форм для моментальной синхронизации введенных данных со свойствами реактивных объектов.

- **Глобальный стейт через Composables:** параметры активной сессии (`useAuth`) и текущей темы оформления (`useTheme`) хранятся в модулях вне дерева компонентов. Любой узел дерева, например, навбар или форма бронирования, может напрямую подписаться на эти данные без необходимости пробрасывать их через цепочку промежуточных компонентов.

3. Практическая реализация ключевых подсистем

3.1. Клиентская маршрутизация и Navigation Guards (src/router/index.js)

В рамках перехода к архитектуре одностраничного приложения (SPA) переходы между экранами были полностью переведены на библиотеку Vue Router 4. В отличие от многостраничной реализации из прошлых работ, где переход по ссылке инициировал новый сетевой запрос к серверу за HTML-документом, роутер перехватывает изменение URL в адресной строке браузера через History API и динамически монтирует нужный компонент в контейнер `<router-view />`.

Конфигурация маршрутов и отложенная загрузка (Lazy Loading)

Чтобы оптимизировать производительность и сократить размер начального JS-бандла, загружаемого клиентом при первом открытии сайта, для всех страниц была применена ленивая подгрузка. При таком подходе сборщик Vite разделяет код приложения на изолированные чанки, а физическая загрузка скрипта страницы инициируется браузером только в момент первого перехода пользователя по соответствующему адресу.

```
JavaScript
import { createRouter, createWebHistory } from 'vue-router'
import { useAuth } from '../composables/useAuth'

const routes = [
  {
    path: '/',
    name: 'home',
    component: () => import('../views/HomeView.vue')
  },
  {
    path: '/tours/:id',
    name: 'tour-details',
    component: () => import('../views/TourDetailsView.vue'),
    props: true
  },
  {
    path: '/dashboard',
    name: 'dashboard',
    component: () => import('../views/DashboardView.vue'),
    meta: { requiresAuth: true }
  },
  {
    path: '/login',
```

```

    name: 'login',
    component: () => import('../views/LoginView.vue'),
    meta: { guestOnly: true }
  },
  {
    path: '/register',
    name: 'register',
    component: () => import('../views/RegisterView.vue'),
    meta: { guestOnly: true }
  },
  {
    path: '/*',
    redirect: '/'
  }
]

```

Дополнительно настроена функция `scrollBehavior`, которая обеспечивает плавную прокрутку к якорю `#catalog` при переходе по ссылке из навигационного меню и автоматический возврат к началу экрана при смене маршрута.

Защита маршрутов через Navigation Guards

Для разграничения прав доступа к экранам сервиса в конфигурации роутера предусмотрены мета-поля `meta`:

- `requiresAuth: true` - флаг для закрытых экранов, требующих обязательного наличия активного досье исследователя (личный кабинет);
- `guestOnly: true` - флаг для экранов авторизации и регистрации, которые не должны быть доступны уже вошедшим пользователям.

Глобальный навигационный барьер реализован в хуке `router.beforeEach` без использования устаревшего колбэка `next()`, что соответствует спецификации актуальных версий Vue Router:

```

JavaScript
router.beforeEach((to, from) => {
  const { isAuthenticated } = useAuth()
  // Попытка зайти в закрытый кабинет без активной сессии
  if (to.meta.requiresAuth && !isAuthenticated.value) {
    return { name: 'login' }
  }

  // Попытка авторизованного пользователя открыть экран входа или
  // регистрации
  if (to.meta.guestOnly && isAuthenticated.value) {
    return { name: 'dashboard' }
  }
})

```

```
}  
})
```

При попытке неавторизованного пользователя напрямую перейти по ссылке `/dashboard` роутер мгновенно прерывает исходную навигацию и выполняет перенаправление на маршрут `login`. В свою очередь, авторизованный кадет при обращении к страницам входа или регистрации автоматически перенаправляется обратно в терминал личного кабинета.

3.2. Модульный сетевой слой на базе Axios (src/api/)

В лабораторной работе № 2 сетевые запросы формировались через самодельный модуль-обертку над Fetch API. При миграции на Vue 3 стек был унифицирован за счет библиотеки Axios. Преимуществами перехода стали автоматическая сериализация данных в формат JSON при отправке, автоматический парсинг входящих ответов сервера и удобный синтаксис работы с промисами.

Конфигурация базового экземпляра Axios

В файле `src/api/instance.js` создан централизованный экземпляр Axios с фиксированным базовым адресом локального сервера базы данных:

```
JavaScript  
import axios from 'axios';  
  
const api = axios.create({  
  baseURL: 'http://localhost:3000',  
  headers: {  
    'Content-Type': 'application/json'  
  }  
});  
  
export default api;
```

Централизация конфигурации в едином инстансе устраняет дублирование настроек сетевого клиента: при смене порта или переносе сервера на удаленный домен достаточно скорректировать переменную `baseURL` в одном месте.

Декомпозиция по доменным модулям

Вся логика взаимодействия с коллекциями `db.json` разделена на три предметных модуля в соответствии с архитектурным стилем REST:

1. **Модуль каталога экспедиций (src/api/tours.js):** инкапсулирует получение полного реестра полетов для каталога главной страницы и точечную выборку конкретного маршрута по строковому идентификатору:

```
JavaScript
```

```
import api from './instance';

export const toursApi = {
  getAll: () => api.get('/tours').then(res => res.data),
  getById: (id) => api.get(`/tours/${id}`).then(res => res.data)
};
```

2. **Модуль учетных записей и досье кадетов (src/api/users.js):** отвечает за поиск исследователя при авторизации, проверку уникальности бортового email при регистрации, запрос актуального профиля по ID и частичную модификацию систем каюты методом PATCH:

```
JavaScript
import api from './instance';

export const usersApi = {
  getByCredentials: (email, password) =>

  api.get(`/users?email=${encodeURIComponent(email)}&password=${encodeURIComponent(password)}`).then(res => res.data),
  getEmail: (email) =>
    api.get(`/users?email=${encodeURIComponent(email)}`).then(res => res.data),
  getById: (id) =>
    api.get(`/users/${id}`).then(res => res.data),
  create: (userData) =>
    api.post('/users', userData).then(res => res.data),
  updateCabinSettings: (id, cabinSettings) =>
    api.patch(`/users/${id}`, { cabinSettings }).then(res => res.data)
};
```

3. **Модуль полетных манифестов (src/api/bookings.js):** обеспечивает фильтрацию записей бронирования по внешнему ключу пользователя `userId` для заполнения бортового журнала в личном кабинете и отправку новых оформленных контрактов методом POST:

```
JavaScript
import api from './instance';

export const bookingsApi = {
  getUserId: (userId) =>
    api.get(`/bookings?userId=${userId}`).then(res => res.data),
  create: (bookingData) =>
```

```
api.post('/bookings', bookingData).then(res => res.data)
};
```

Разделение эндпоинтов на специализированные модули полностью изолировало компоненты пользовательского интерфейса от прямых сетевых вызовов: компоненты не знают об URL-структуре API и работают исключительно с типизированными методами модулей, получая чистые данные из тела ответов `res.data`.

3.3. Изоляция бизнес-логики в Composables (src/composables/)

В лабораторной использовался паттерн composables - функции, инкапсулирующие состояние и логику на базе Composition API. В проекте NovaTransit повторяющиеся сценарии вынесены в три модуля, что позволило полностью избавить компоненты от тяжелой служебной логики и сделать их код декларативным.

Управление темами оформления (src/composables/useTheme.js)

Модуль отвечает за переключение между темной темой *Eclipse* и светлой темой *Nova*. Он обеспечивает трехуровневый алгоритм выбора активного режима:

1. Сначала проверяется сохраненное значение в `localStorage` по ключу `nova_theme`.
2. Если пользователь еще не переключал тему вручную, считываются системные предпочтения операционной системы через `window.matchMedia('(prefers-color-scheme: light)').`
3. При отсутствии системных предпочтений активируется базовая темная тема.

```
JavaScript
import { ref } from 'vue';

const THEME_STORAGE_KEY = 'nova_theme';
const currentTheme = ref('dark');

function applyTheme(theme, save = true) {
  currentTheme.value = theme;
  document.documentElement.setAttribute('data-theme', theme);
  document.documentElement.setAttribute('data-bs-theme', theme);
  if (save) {
    localStorage.setItem(THEME_STORAGE_KEY, theme);
  }
}

export function useTheme() {
  const toggleTheme = () => {
    const nextTheme = currentTheme.value === 'light' ? 'dark' : 'light';
    applyTheme(nextTheme, true);
  };
}
```

```
};

return { currentTheme, toggleTheme };
}
```

Скрипт параллельно обновляет атрибуты `data-theme` для кастомных CSS-переменных проекта и `data-bs-theme` для встроенных стилей Bootstrap, обеспечивая синхронную смену фона, прозрачности стекла и цвета текста. Также в модуль встроен слушатель системных событий, автоматически меняющий режим на лету, если пользователь переключил тему в своей ОС.

Реактивное управление сессией кадета (`src/composables/useAuth.js`)

Вся клиентская аутентификация и контроль сессии изолированы внутри `useAuth`. Глобальное состояние текущего исследователя хранится в реактивной переменной `user`, объявленной вне экспортируемой функции, благодаря чему все компоненты приложения получают доступ к единому источнику данных:

- `isAuthenticated`: вычисляемое свойство (`computed`), сигнализирующее о наличии авторизованной сессии.
- `login(email, password)`: обращается к методу `usersApi.getByCredentials`, проверяет существование исследователя, сохраняет профиль в `user` и синхронизирует его с `localStorage`.
- `register(name, email, password)`: проверяет уникальность бортового email через `usersApi.getByEmail`. При отсутствии совпадений генерирует случайный шифр допуска `NT-XXXX-X`, дефолтный аватар сервиса UI-Avatars и первичную конфигурацию каюты, после чего сохраняет нового пользователя на сервере методом POST.
- `updateCabinSetting(key, value)`: отвечает за частичное обновление настроек жизнеобеспечения каюты. Функция модифицирует реактивный объект `user.value.cabinSettings` и отправляет запрос `PATCH /users/:id` на сервер базы данных.

```
JavaScript
const updateCabinSetting = async (key, value) => {
  if (!user.value) return;
  const updatedSettings = {
    ...user.value.cabinSettings,
    [key]: value
  };
  const updatedUser = await usersApi.updateCabinSettings(user.value.id, updatedSettings);
  setSession(updatedUser);
};
```

Отказоустойчивый шлюз NASA APOD ([src/composables/useNasaApod.js](#))

Для интеграции со сторонним REST API космического агентства NASA разработан composable [useNasaApod](#). В composable реализована трехступенчатая отказоустойчивая цепочка:

1. **Кэш сессии:** при вызове функции [fetchApod](#) сначала проверяется [sessionStorage](#). Если снимок дня уже запрашивался в текущей вкладке, данные мгновенно берутся из памяти, что предотвращает лишний расход квоты API при переходах по роутеру.
2. **Прямой и прокси-запрос:** клиент отправляет вызов к официальному эндпоинту [api.nasa.gov](#). Если сервис возвращает ошибку (статусы 403, 429 или сбой CORS), скрипт перехватывает управление и обращается к резервному CORS-прокси.
3. **Автономный оффлайн-кэш:** при полном отсутствии интернет-соединения блок [catch](#) присваивает переменной [apodData](#) резервный объект с локальной панорамой Земли [assets/images/earth-1.jpg](#), а переменной [isOffline](#) - значение [true](#). Это исключает падение страницы и информирует пользователя о переходе в автономный режим через бейдж [Offline Cache](#).

3.4. Интерактивные интерфейсы и реактивные вычисления

Благодаря возможностям Vue 3 интерфейс сервиса перестал требовать ручных манипуляций с DOM-деревом через методы [querySelector](#) или [classList](#), которые использовались в старых скриптах.

Мгновенная фильтрация каталога через [computed](#)

На главной странице [HomeView.vue](#) фильтрация экспедиций реализована на клиенте с помощью вычисляемого свойства [filteredTours](#). Параметры формы (направление, длительность полета и класс миссии) связаны с реактивным объектом [filters](#).

```
JavaScript
const filteredTours = computed(() => {
  return tours.value.filter(tour => {
    const matchDest = !filters.destination || tour.destination ===
filters.destination;
    const matchDur = !filters.duration || tour.duration ===
filters.duration;
    const matchBud = !filters.budget || tour.budget === filters.budget;
    return matchDest && matchDur && matchBud;
  });
});
```

В отличие от LP2, где карточки скрывались добавлением CSS-класса [d-none](#) вручную через цикл, вычисляемое свойство Vue автоматически отслеживает изменение любого поля в форме поиска и за доли миллисекунды возвращает отфильтрованный массив. Директива [v-for](#) сама перерисовывает карточки в сетке, а условие [filteredTours.length === 0](#) мгновенно выводит информационную плашку «Экспедиций не найдено» при отсутствии совпадений.

Динамический слайдер фотохроники (**TourGallery.vue**)

Вместо использования громоздких и нестабильных JS-скриптов карусели Bootstrap компонент фотохроники переведен на нативную логику Vue. Слайдер состоит из гибкой ленты `slider-track`, содержащей все кадры миссии, и внешнего контейнера со свойством `overflow: hidden`.

Смещение кадров реализовано через привязку inline-стилей к реактивному индексу `currentSlide`:

```
HTML
<div
  class="slider-track d-flex h-100"
  :style="{ transform: `translateX(-${currentSlide * 100}%)` }"
>
```

В CSS ленте задано свойство `will-change: transform` и кубическая кривая Безье `cubic-bezier(0.25, 1, 0.5, 1)`, что переносит отрисовку анимации на графический процессор (GPU) и обеспечивает плавный сдвиг кадров без просадки частоты кадров. Дополнительно настроен таймер автопрокрутки (`setInterval`), который переключает снимки каждые 5 секунд. По событиям `@mouseenter` и `@mouseleave` прокрутка автоматически ставится на паузу при чтении описания и возобновляется при уходе курсора.

Модальное окно бронирования и плавная анимация (**BookingModal.vue**)

Диалоговое окно оформления контракта на полет адаптировано под реактивную форму с динамическим расчетом стоимости. При открытии окна поля ФИО и бортовой почты автоматически заполняются данными из профиля авторизованного исследователя.

Расчет финальной стоимости полета реализован через вычисляемое свойство `totalPrice`, которое моментально реагирует на состояние чекбоксов дополнительных модулей:

```
JavaScript
const totalPrice = computed(() => {
  let total = props.tour.price || 0;
  if (form.optEva) total += 150000;
  if (form.optPhoto) total += 50000;
  return total;
});
```

Для визуального оформления модального окна использован встроенный компонент `<Transition name="modal-fade">`. Это позволило избавиться от резкого появления окна: полупрозрачная подложка плавно растворяется через изменение прозрачности (`opacity`), а сам диалоговый блок опускается сверху с легким масштабированием (`transform: translateY(-25px) scale(0.96)`), создавая ощущение живого интерфейса.

Блок тумблеров жизнеобеспечения и Optimistic UI

В личном кабинете исследователя панель управления каютой (**CabinControls.vue**) взаимодействует с профилем пользователя через паттерн Optimistic UI.

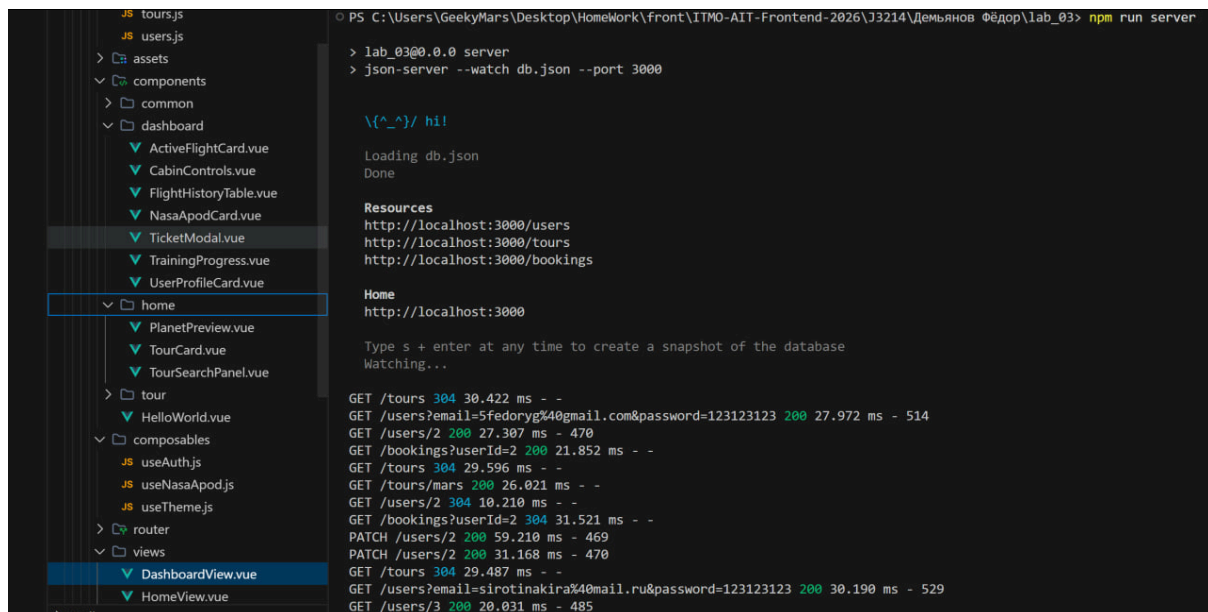
При щелчке по тумблеру (гравитация, гипоаллергенное питание или автозатемнение иллюминаторов) приложение не ждет подтверждения от сервера. Компонент сразу же генерирует событие **notify**, которое запускает всплывающий информационный блок бортового компьютера (Bootstrap Toast) с новым статусом подсистемы. Параллельно в фоновом режиме выполняется асинхронный вызов **updateCabinSetting**, отправляющий запрос **PATCH** в файл **db.json**. Если соединение с сервером прерывается, блок **catch** информирует кадета о сетевом сбое, сохраняя целостность данных. Сообщение тоста анимировано через **<Transition name="toast-fade">** и автоматически исчезает через 3.5 секунды.

4. Результаты тестирования

Работоспособность разработанного веб-приложения проверена в ходе тестирования пользовательских сценариев.

4.1. Развертывание рабочей среды и запуск сервисов

Приложение запускается в двух параллельных процессах: локальный dev-сервер сборщика Vite (порт 5173) и Mock-сервер **json-server** (порт 3000), обслуживающий базу данных **db.json**.



```
JS tours.js
JS users.js
> assets
> components
> common
> dashboard
  > ActiveFlightCard.vue
  > CabinControls.vue
  > FlightHistoryTable.vue
  > NasaApodCard.vue
  > TicketModal.vue
  > TrainingProgress.vue
  > UserProfileCard.vue
> home
  > PlanetPreview.vue
  > TourCard.vue
  > TourSearchPanel.vue
> tour
  > HelloWorld.vue
> composables
  > useAuth.js
  > useNasaApod.js
  > useTheme.js
> router
> views
  > DashboardView.vue
  > HomeView.vue
> Outline

PS C:\Users\GeekyMars\Desktop\HomeWork\front\ITMO-AIT-Frontend-2026\J3214\Демьянов Фёдор\lab_03> npm run server

> lab_03@0.0.0 server
> json-server --watch db.json --port 3000

\(^_)/ hi!

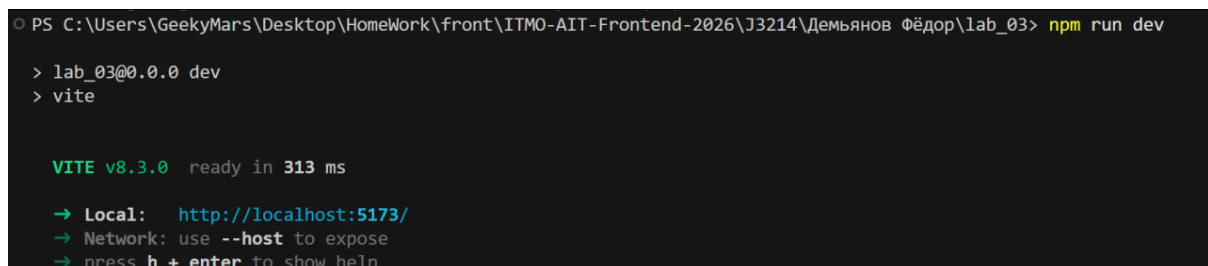
Loading db.json
Done

Resources
http://localhost:3000/users
http://localhost:3000/tours
http://localhost:3000/bookings

Home
http://localhost:3000

Type s + enter at any time to create a snapshot of the database
Watching...

GET /tours 304 30.422 ms - -
GET /users?email=5fedoryg%40gmail.com&password=123123123 200 27.972 ms - 514
GET /users/2 200 27.307 ms - 470
GET /bookings?userId=2 200 21.852 ms - -
GET /tours 304 29.596 ms - -
GET /tours/mars 200 26.021 ms - -
GET /users/2 304 10.210 ms - -
GET /bookings?userId=2 304 31.521 ms - -
PATCH /users/2 200 59.210 ms - 469
PATCH /users/2 200 31.168 ms - 470
GET /tours 304 29.487 ms - -
GET /users?email=sirotinakira%40mail.ru&password=123123123 200 30.190 ms - 529
GET /users/3 200 20.031 ms - 485
```



```
PS C:\Users\GeekyMars\Desktop\HomeWork\front\ITMO-AIT-Frontend-2026\J3214\Демьянов Фёдор\lab_03> npm run dev

> lab_03@0.0.0 dev
> vite

VITE v8.3.0 ready in 313 ms

→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Рисунок 1 - Терминалы сборщика Vite, локального сервера базы данных и файловая структура проекта

4.2. Регистрация нового исследователя и контроль доступа (RegisterView.vue, LoginView.vue)

При переходе на маршрут `/register` проверяется работа клиентской валидации: при отправке пустой формы поля подсвечиваются ошибками. После ввода валидного имени, почты, ключа доступа от 6 символов и подтверждения регламента полетов метод `register (useAuth)` выполняет запрос `POST /users` в базу `db.json`. Система присваивает исследователю код `NT-XXXX-X` и выполняет редирект в защищенный кабинет `/dashboard`.

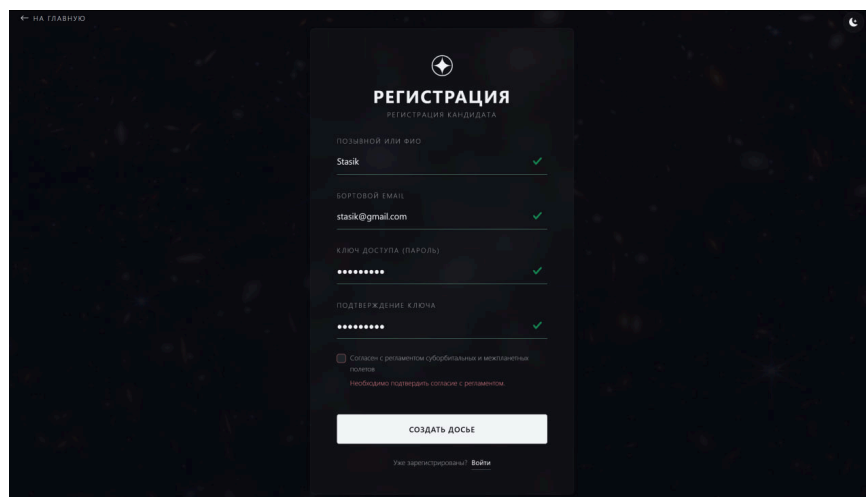


Рисунок 2 - Терминал регистрации исследователя с валидацией полей формы

4.3. Интерактивный промо-раздел Него и выбор направления (HomeView.vue, PlanetPreview.vue)

На главном экране в панели поиска выбирается направление «Марс». Компонент `PlanetPreview` плавно сменяет текстуру на марсианскую поверхность и пересчитывает гравитационные показатели (0.38 G). При выборе класса миссии «VIP Explorer» панель поиска подсвечивается золотистым свечением.

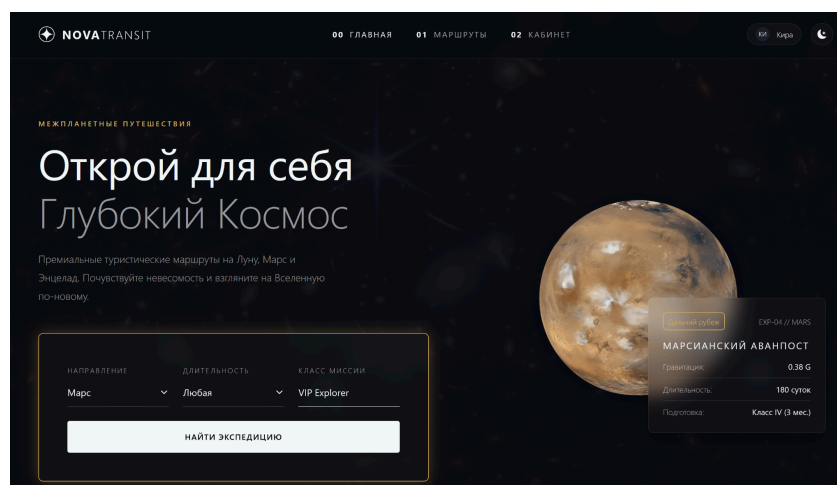


Рисунок 3 - Интерактивный промо-блок Него с моделью Марса и адаптивной панелью фильтрации

4.4. Клиентская фильтрация каталога миссий (HomeView.vue, TourCard.vue)

По нажатию кнопки «Найти экспедицию» страница плавно скроллится к реестру полетов. Вычисляемое свойство `filteredTours` мгновенно отсекает неподходящие туры без отправки запросов на сервер, оставляя в сетке карточку экспедиции «Марсианский транзит (EXP-04)» с актуальной стоимостью и параметрами длительности.

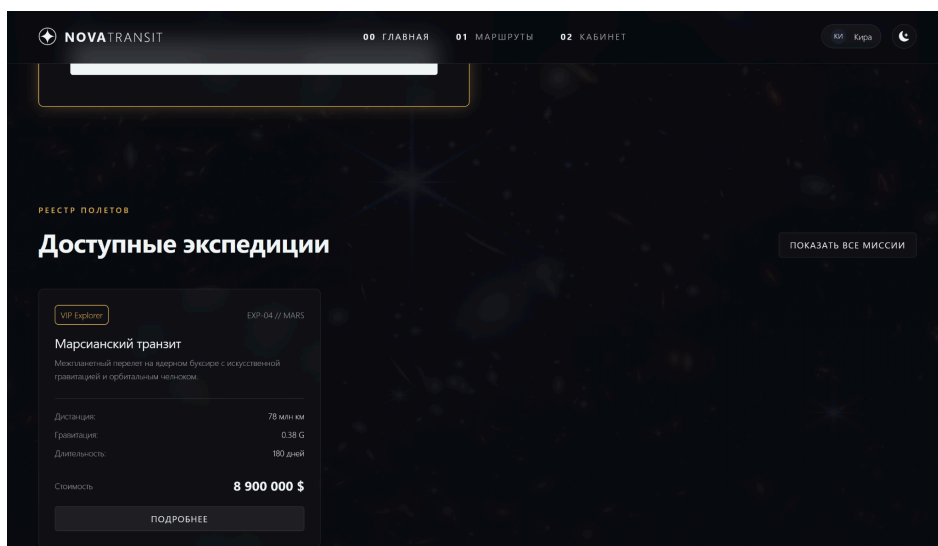


Рисунок 4 - Реестр доступных полетов после применения клиентских фильтров

4.5. Анализ параметров полета и фотохроника (TourDetailsView.vue, TourGallery.vue)

При переходе по адресу `/tours/mars` компонент обращается к `toursApi.getById('mars')`. Слайдер `TourGallery` выполняет горизонтальный сдвиг ленты кадров через `transform: translateX()` с аппаратным ускорением GPU, автоматически сменяя фотографии раз в 5 секунд и приостанавливая таймер при наведении курсора.

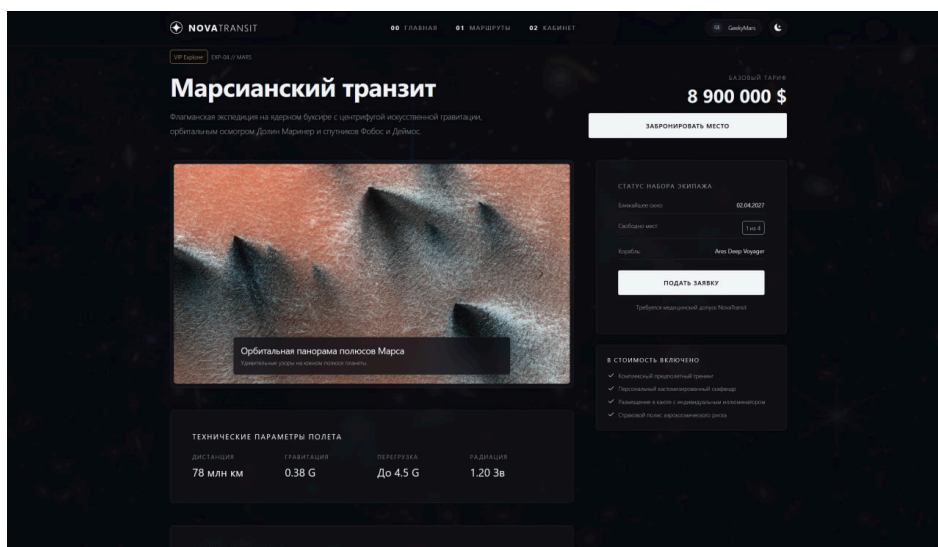


Рисунок 5 - Детальный экран экспедиции с фотохроникой и физико-техническими характеристиками

4.6. Реактивное бронирование и расчет сметы (BookingModal.vue)

Клик по кнопке «Забронировать место» открывает модальное окно через компонент `<Transition>`. Поля ФИО и email предзаполняются из данных сессии кадета. При выборе чекбоксов шлюзования и/или внешней съемки 8K вычисляемое свойство `totalPrice` моментально пересчитывает сумму. Подтверждение заявки отправляет контракт в `POST /bookings`.

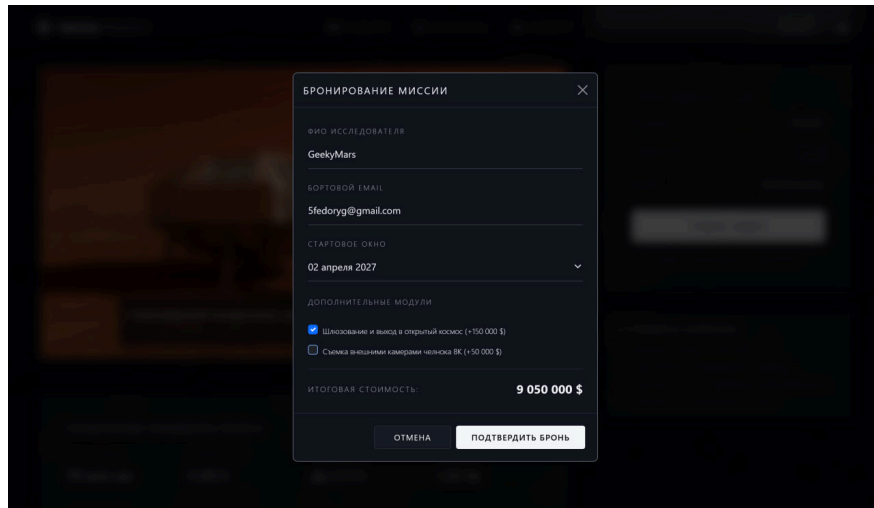


Рисунок 6 - Модальное окно бронирования миссии с динамическим расчетом стоимости контракта

4.7. Личный кабинет исследователя и посадочный талон (DashboardView.vue, ActiveFlightCard.vue)

В закрытом терминале `/dashboard` блок `ActiveFlightCard` отображает свежеоформленную бронь: маршрутную дугу `EAR → MARS` с иконкой челнока, шлюз А-12 и статус «Ожидает старта». Рядом выводится карточка исследователя с аватаром, персональным шифром допуска и шкалами выполнения предполетных нормативов.

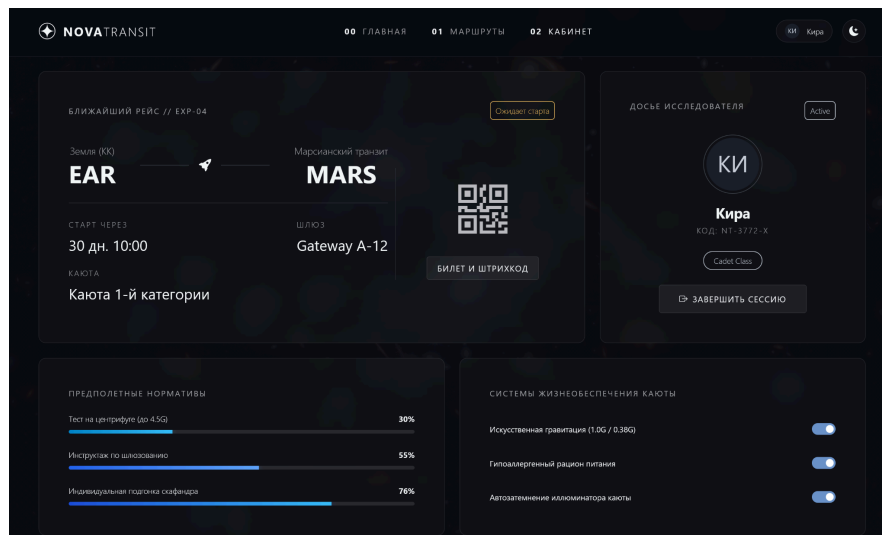


Рисунок 7 - Личный кабинет исследователя с активным полетным купоном и досье кадета

4.8. Телеметрия жизнеобеспечения каюты (CabinControls.vue)

При клике по тумблеру гравитации интерфейс реагирует мгновенно: в правом нижнем углу плавно всплывает системный Toast бортового компьютера «Режим гравитации: статус изменен на [ДЕАКТИВИРОВАН]». В фоновом режиме на сервер отправляется запрос `PATCH /users/:id`, обновляющий булево значение поля `gravity` в файле `db.json`.

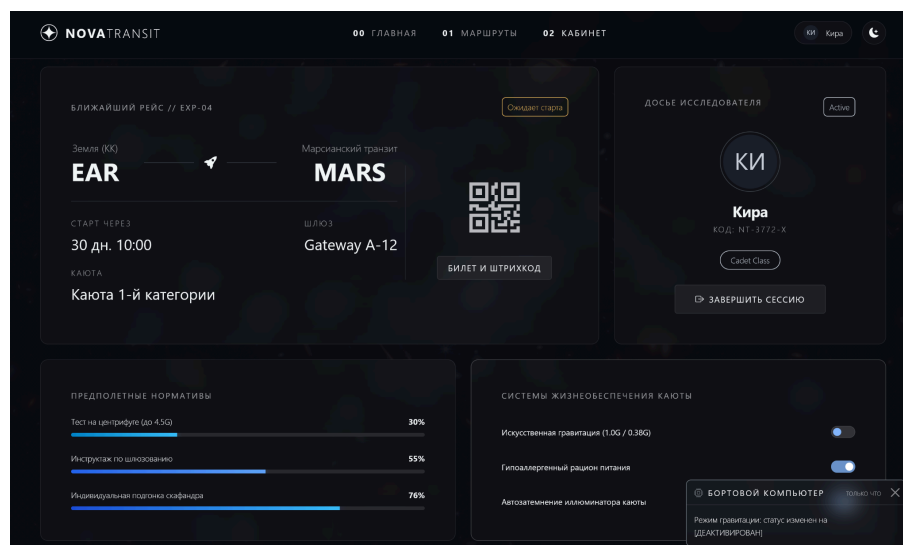


Рисунок 8 - Управление системами каюты кадета и системное Toast-уведомление

4.9. Внешний телеметрический шлюз NASA APOD (NasaApodCard.vue, useNasaApod.js)

Виджет внешнего шлюза обращается к API NASA через composable `useNasaApod`. Компонент отображает актуальный снимок дня в высоком разрешении, авторскую экспликацию и статус `Public API Online`. Полученные данные кэшируются в `sessionStorage`, что исключает дублирование запросов к внешнему шлюзу при переходах по роутеру.

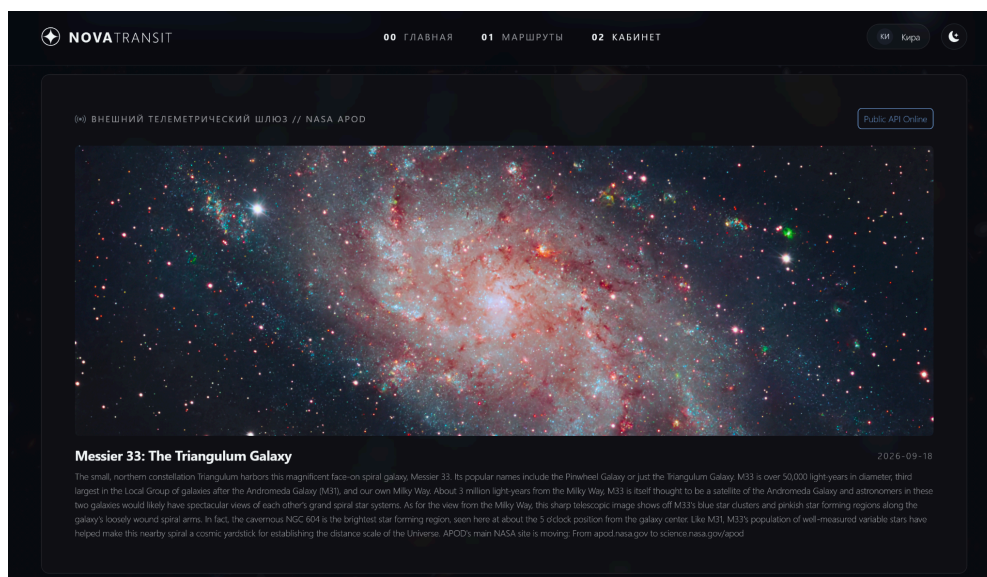


Рисунок 9 - Внешний телеметрический шлюз интеграции с NASA APOD API

4.10. Альтернативная тема оформления Nova (Axios)

Нажатие на кнопку-тумблер темы в навигационной панели активирует метод `toggleTheme` из `useTheme`: тегу `<html>` присваивается атрибут `data-theme="light"`, фоны панелей и цвет типографики инвертируются в светлую гамму.

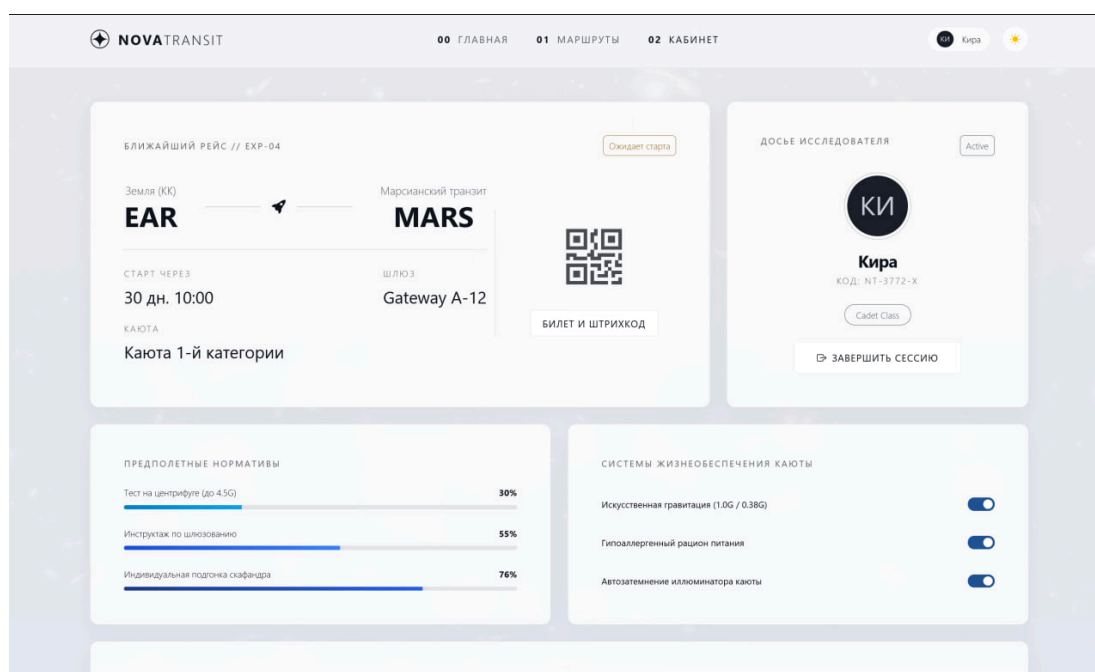


Рисунок 4.10 - Светлая тема оформления Nova и мониторинг сетевых запросов в консоли разработчика

4.11. Сетевая активность (useTheme.js, Axios)

В панели инструментов разработчика (вкладка «Сеть») зафиксирована корректная обработка сетевых запросов при загрузке главной страницы с активной сессией пользователя «Кира»:

- **Запрос каталога через Axios:** модуль `toursApi` успешно выполнил запрос `GET http://localhost:3000/tours` к серверу базы данных. В столбце «Инициатор» зафиксирован вызов через `axios.js (xhr)`, сервер вернул статус `304 Not Modified` с размером полезной нагрузки 4,12 КБ из кэша.
- **Загрузка аватара пользователя:** компонент навигации выполнил запрос к внешнему сервису `ui-avatars.com` (`GET /api/?name=Кира...`, статус 200 OK) для отображения инициалов авторизованного исследователя.
- **Журнал консоли:** встроенная консоль фиксирует стабильное подключение сборщика Vite (`connected from window, ping successful`) без ошибок выполнения и предупреждений Vue.

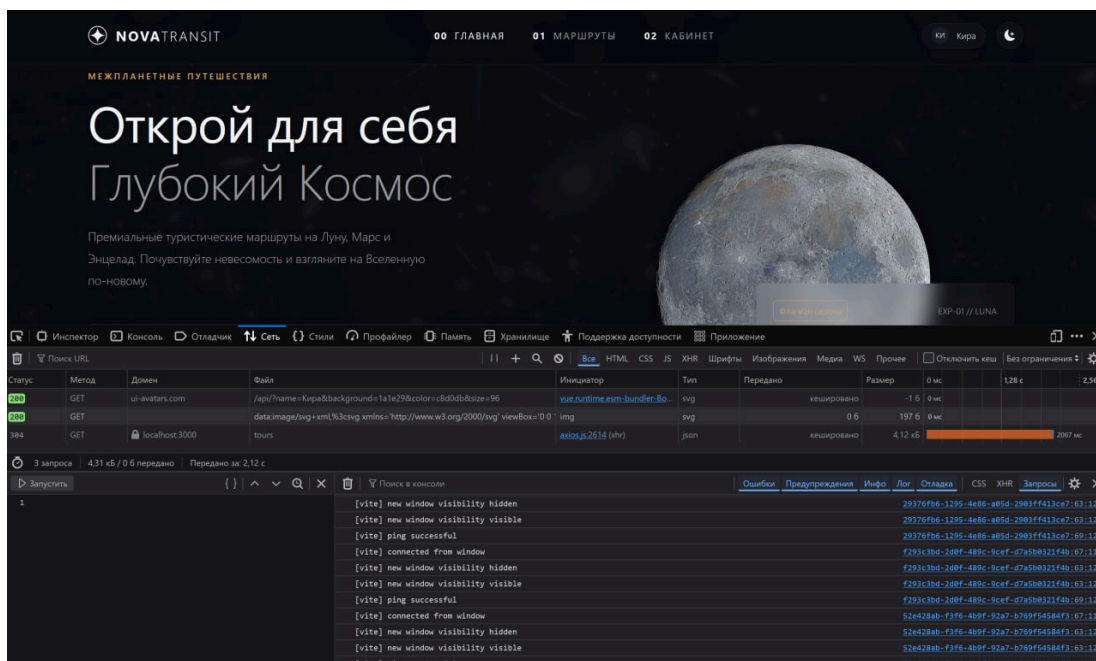


Рисунок 4.11 - Сетевые запросы Axios к локальному REST API и генерация аватара сессии исследователя

5. Заключение и рефлексия

Перевод NovaTransit на Vue 3 стал отличным финалом для всей серии заданий. Работать над этим проектом на протяжении всех трех лабораторных работ было по-настоящему интересно. Было здорово наблюдать, как сайт развивался шаг за шагом: от обычной статичной верстки и бесконечного поиска элементов в JS до быстрого, аккуратного приложения, разбитого на удобные блоки.

Подход Vue с реактивными переменными оказался намного удобнее и надежнее прежнего кода. Больше не приходится вручную искать теги через `querySelector`, склеивать куски HTML в строках или вручную переключать классы видимости. Экран сам обновляет нужные места, как только меняются данные.

Вынос логики в отдельные вспомогательные модули помог навести порядок и избавил страницы от лишней нагрузки. В итоге связка Vite, роутера и Axios дала понятный практический опыт.